

Object detection of Spices in Warehouses

Sugunadevi Palaniappan
Intelligent Systems, Technische Hochschule Ulm

1 Introduction

I tried to follow CRISP Model for my implementation. The following phases are followed here.

1. Business Understanding
2. Data Understanding
3. Modeling
4. Evaluation
5. Deployment

Business Understanding

Warehouse Automation is primary focus for many eCommerce giants such as Amazon, eBay. High labour costs, inaccurate inventory, redundant processes are major problems in warehouses. For complete automation, highly specialized equipment must be able to identify all objects in warehouses on their own. Models could be trained for generic objects such as apples, oranges, bottles with available vast number of datasets. It is pretty interesting to perform detection of specific objects such as spices. My experience on this journey of spice detection is documented here.

Data Understanding

It's understood that there is no available image datasets for spices in online, so decided to create on own. Choosing a list of spices is difficult as most of them are very small and in powders. Gathered some spices images and got familiar with the spices data. Since powders are difficult to detect, packets and boxes containing those spices are used. Then, spices which are larger in size are used for detection.

2 Data Preparation

A vast amount of time is spent on this phase than expected. Initially datasets are not sufficient, datas are collected and added for each iteration. Since spices are small in size and with small datasets, it's not detected properly. So, some spices of 'smaller size and similar colours are removed. In my case, had problems with packets. Since, spices are very minute, I thought of putting it into packets.

Dataset Collection:

The dataset was created by collecting different types of spices namely cardamom, bayleaf, asofoetida and turmeric. The images of each class were collected in various brightness, contrasts, and angles. The collected images consist of one or more of the above four classes. In first attempt, a total of 120 images are collected, among which 20 each are the images of cardamom, bayleaf, asofoetida and turmeric, and the remaining 40 images consists of all the three objects. Later, 216 images are added to the dataset.

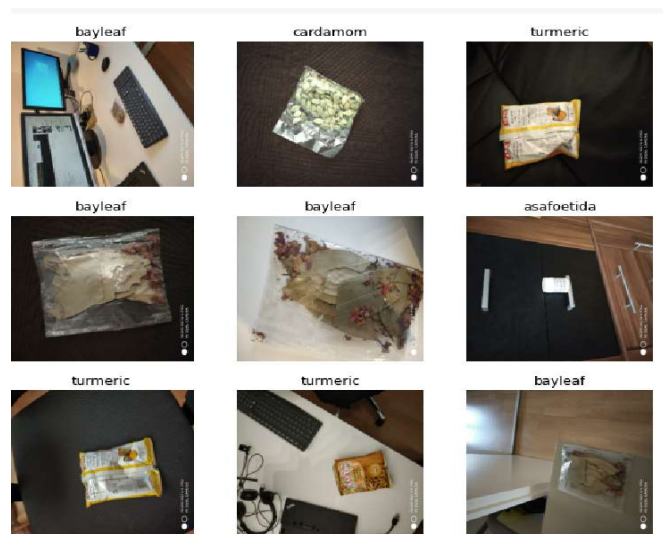


Figure 1: dataset collection

Since large number of images are required for each class to train the deep learning neural network, data augmentation is done.



Figure 2: Augmentation

Dataset Labelling:

Labelling the objects manually by drawing a rectangle box is called as Annotation. The information about the location and dimension of the object could be given to the Model by Annotation. During my experiments, I labelled the entire images before resizing. After annotation, dimensions have changed. So, the entire dataset was rescaled and once again labelled. Later, I learned that Pre-processing steps such as resizing the image is to be done before performing annotation, because location and dimensions of the object varies after resizing.

The application named labellmg is used for the annotation of cardamom, bayleaf, asofoetida and turmeric. labellmg consists of several functionalities including creating a rectangle box, labelling objects, and saving it in xml format.

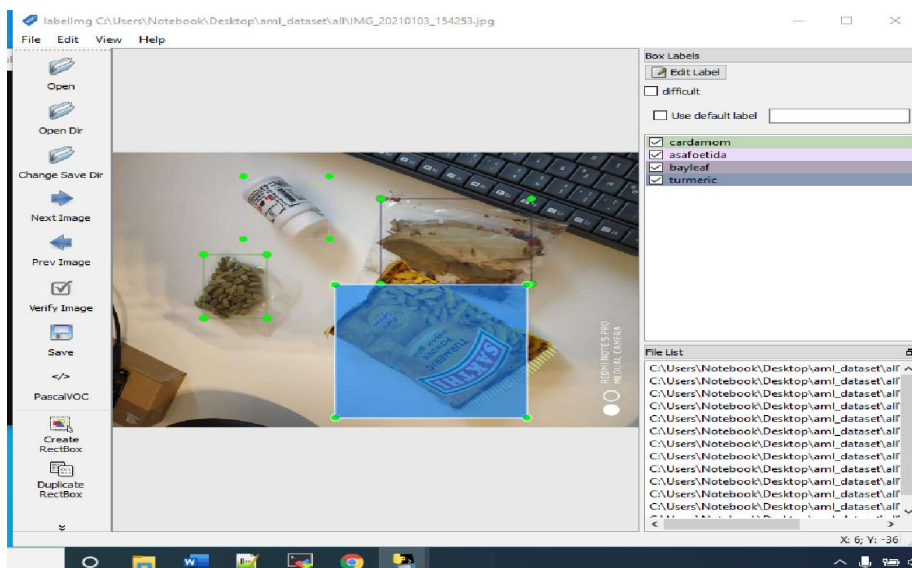


Figure 3: labelling with Labellmg tool

In figure 3, annotated spices in labellmg tool are displayed. Green rectangles are annotation of spices which are labelled by their classes. Once annotation is done, it is saved as xml files which gives information about the location and dimension of labelled objects.

After the completion of annotation, all generated .xml files from the training sample is converted to train_CSV file and the test sample is converted to test_CSV file, which could be further fed to the training of the model.

Data Pre-processing:

Since dimensions are changing after labelling, I resized it initially and then labelled it. Resizing is done using a python script. Pre-processing of input images are done with the help of transforms module present in Torchvision module. Torchvision library has a function named compose(), where various image pre-processing could be grouped. Colour adjustments ,image is converted to PyTorch Tensor data type and the normalization is done.

Python Imaging Library (PIL) offers many procedures for image manipulation. So, the input images are transformed to PILImage

```
custom_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.ColorJitter(saturation=0.3),
    transforms.ToTensor(),  utils.normalize_transform())])
```

Later, some samples with bad brightness and images with noise were removed.

3. Modeling

Framework:

Pre-trained models such as Resnet 50 are trained on large datasets like ImageNet. Instead of training from scratch, weights, and biases of pretrained Resnet50 model is used here. This model takes input as image and predict its class.

Model Inference process includes reading the input image, performing transformations on image such as resize, normalization, use the pre-trained weights to find out the output vector, based on scores obtained, predictions are displayed.

Various changes have been made to the default configuration files of the Faster R-CNN provided by PyTorch API named detecto, such as dataset, number of classes to be trained, batch size and label map.

Training Procedure:

The main idea is to detect spices. The model is trained with training samples from objects such as cardamom, bayleaf, asofoetida and turmeric. Training samples are divided into training dataset and test data set. The train dataset is used to extract the significant features and the test dataset is used to evaluate the features and to calculate the losses. Number of classes to be

trained is set to 4 classes, since the model being trained is expected to detect four classes namely cardamom, bayleaf, asofoetida and turmeric

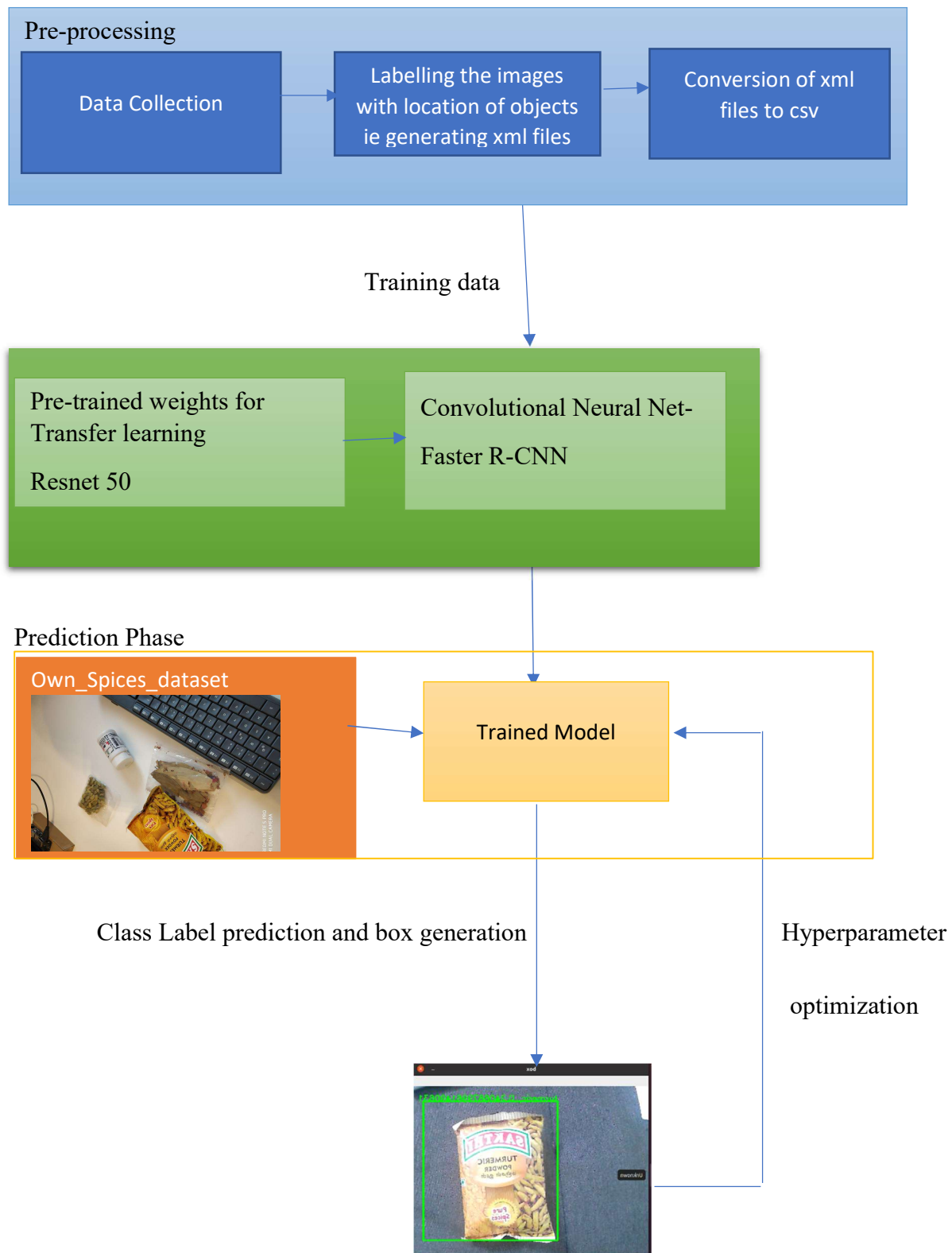


Figure 4: Final Implementation workflow

4. Evaluation

Results from existing models

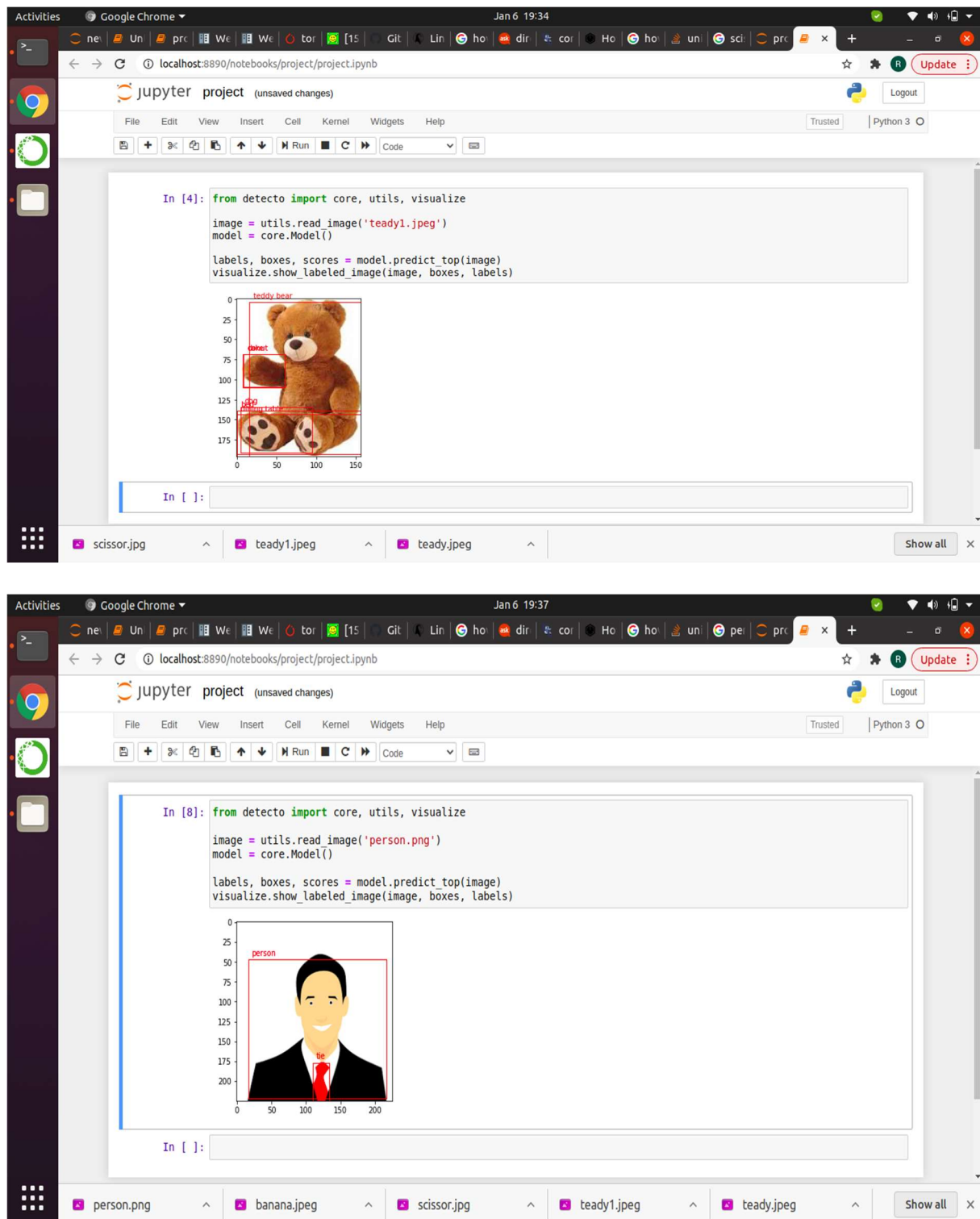


Figure 5: Implementation of existing model

Working on own customized spice model:

Experiment 1 with tensorflow:

Initially, i started to implement in tensorflow. When I tried to build the customized model on existing model, I couldn't make it even I tried it for many days.

Experiment 2 with keras:

Then, performed basic classification task in keras. After several iterations, it worked good. The reports of those classification are shown here.

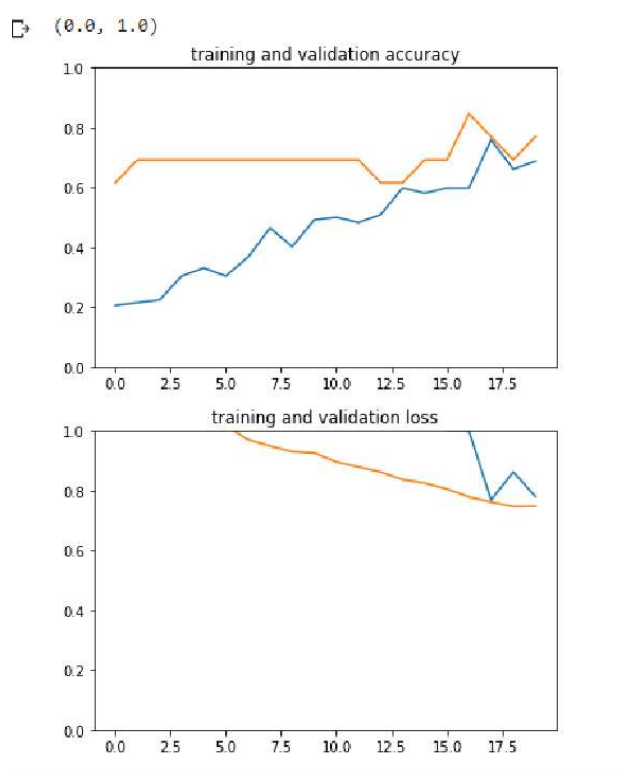


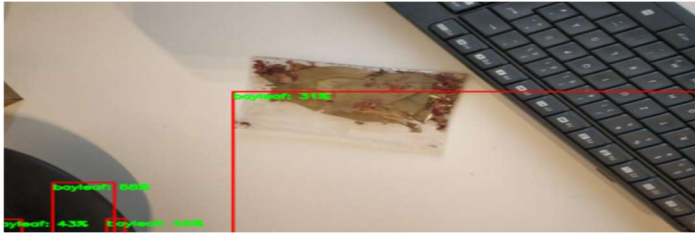
Figure 6: Model Evaluation

	precision	recall	f1-score	support
0	0.80	1.00	0.89	4
1	0.67	1.00	0.80	4
2	1.00	0.25	0.40	4
3	0.75	0.75	0.75	4
accuracy			0.75	16
macro avg	0.80	0.75	0.71	16
weighted avg	0.80	0.75	0.71	16

Figure 7: 'Classification Report on initial analysis

Experiment 3 to draw bounding boxes on live objects:

Then, in addition to classification, tried to include bounding boxes to the objects in images. Real time detection using opencv is tried for first time. The functions for region proposal and non-maxima suppression doesn't worked well. Model is not efficient.



Experiment 4 in Pytorch:

Then, tried to implement object detectors such as Faster R-CNN, SSD or Yolo. On last hope, I found a package detecto and worked on it. Results of those implementation'is below.

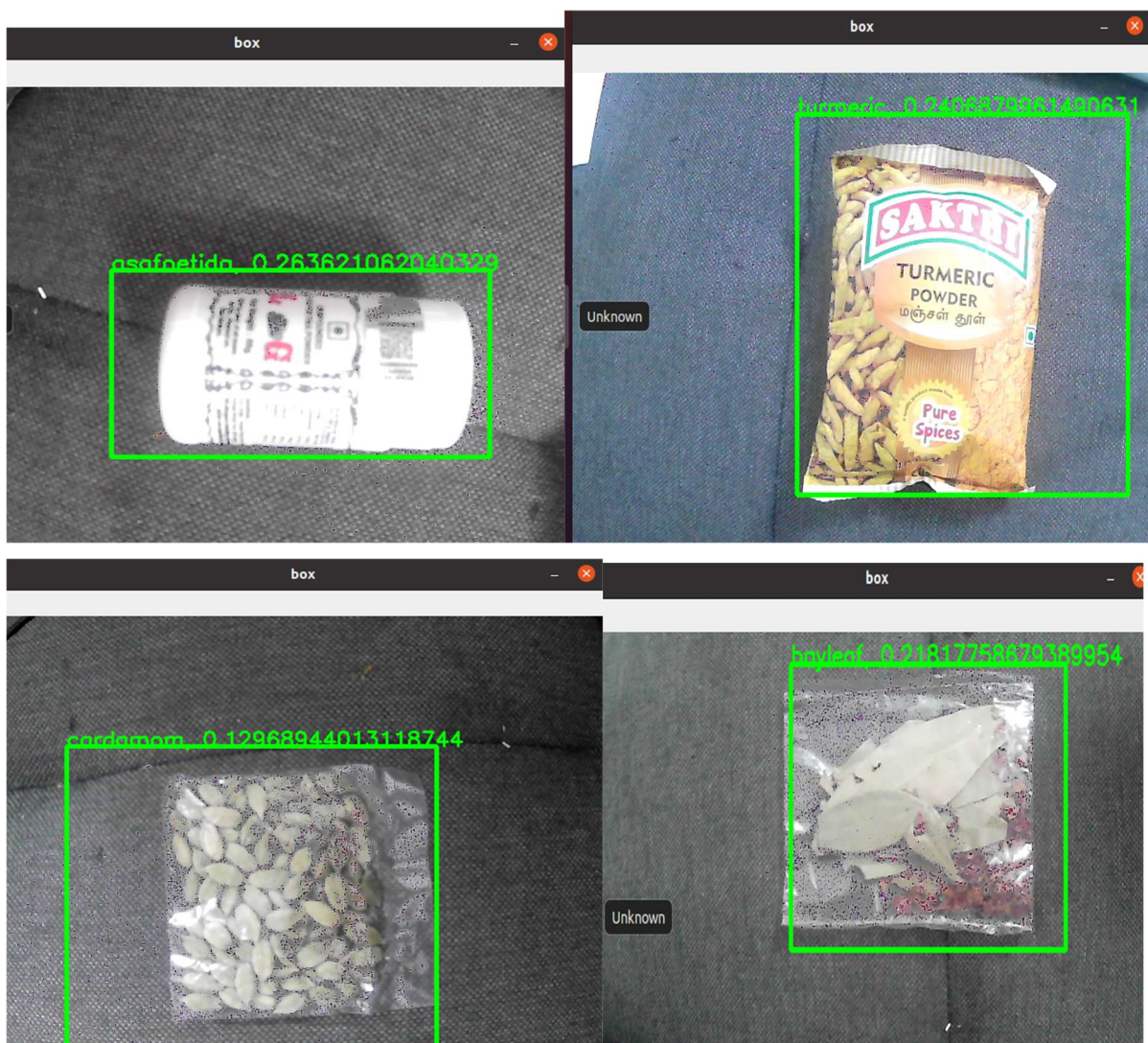


Figure 8: Implementation of customized model

Loss curve

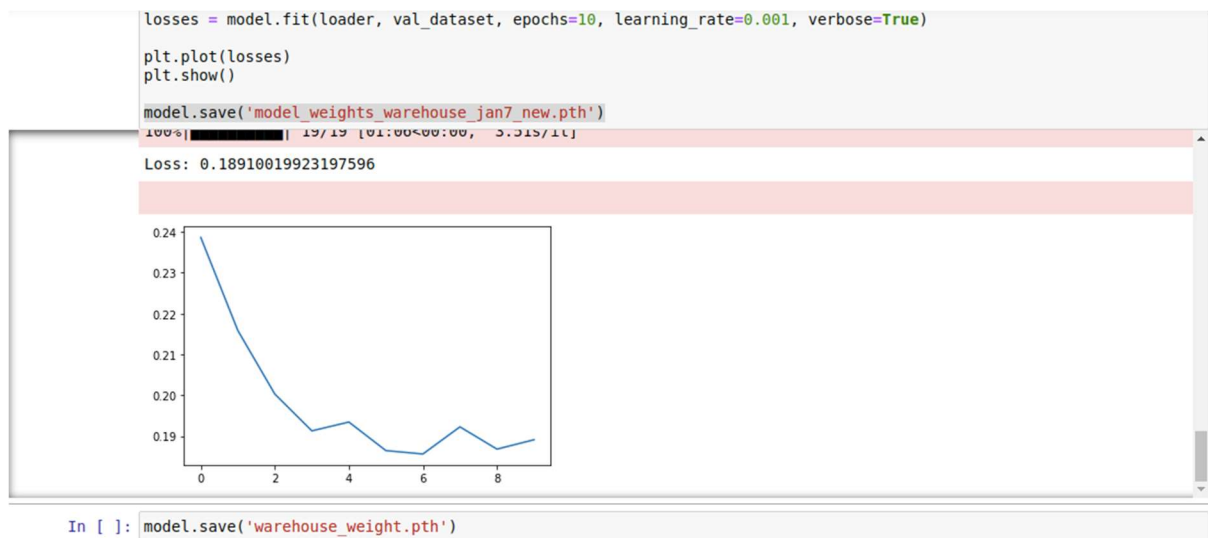


Figure 9: Loss Analysis

In final predictions, glaring of packets on lights created a problem. Difference between turmeric bunch and the cardamom bunch is little similar, it is also a problem. So, I learned that objects should not be packed in covers as it reduces the learning rate. I trained with several optimized hyperparameters. Unfortunately, I didn't save those notebooks, if possible, I upload those saved weights in bwsync.

Working with Nano:

Initial start:

When I tried to write Nano image on SD card with Etcher, my SD card got corrupted. When I looked it in internet, I saw it's a common problem. After lots of hardships, I managed to write the image on Linux environment.

Fan Execution:

Since I am new to such embedded device, it took some time to get familiar. Building fan, making the fan run by executing the given scripts made fun.

Docker execution:

Created a docker image, but the inbuilt python versions are not compatible with detecto module. I tried to create a own image and mount all the installed packages into it, in last minute, I faced many compatibility problems

Nano Execution Problems:

In Jetson Nano, after booting first time with the image, I followed the tutorial <https://www.pyimagesearch.com/2019/05/06/getting-started-with-the-nvidia-jetson-nano/> to install the basic packages required.

Issues with OpenCV:

As a first step, I tried to install openCV with pip using "pip3 install opencv-python". But it resulted in error while importing cv2. Then, I read in Nvidia developer forum that it is recommended to install OpenCV from source and it solved the problem.

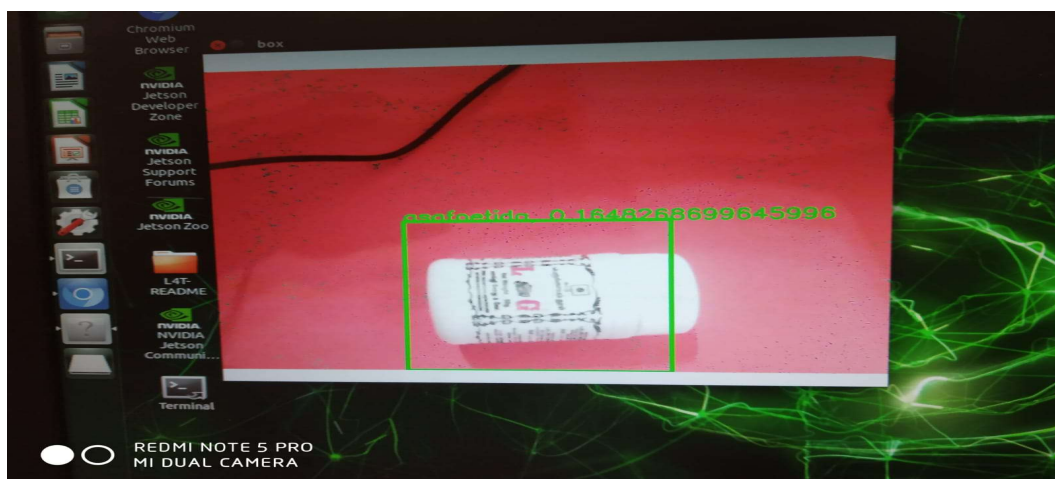
Issues faced with Pytorch:

Initially, I installed PyTorch v1.5.0. But while importing numpy/torch/cv2 packages in python3, it resulted in segmentation fault. Then, I found out from Nvidia developer forum that the version of pytorch could be the problem and it was stated that PyTorch version $\geq$ v1.6.0 need to be used. After upgrading, the PyTorch package worked fine.

Issues with Detecto:

Once I installed Detecto package, importing either of detecto/cv2/numpy packages in python3 resulted in segmentation fault. Later, I found that while installing detecto, numpy package automatically get updated to v1.19.5 which seem to be incompatible with other packages. Then, I re-installed numpy v1.16 and finally all the packages worked fine.

Results after executing on Jetson Nano:





5. Deployment

A summarized overview of Spices Detection is documented here and real time object detection is to be presented.

6. Conclusion

The results look not promising provided with the extremely small set of training samples. It can be improved further for sure if more time is spent. Detection of Indian spices is little challenging because of their smaller object size. However, there are possibilities to improve its performance. Some of the possibilities is to increase the training samples which is extremely small in the developed model and using other pre-trained models.